

操作系统

第二章 进程管理

第二部分 (2) 系统调用 (下)

同济大学计算机系



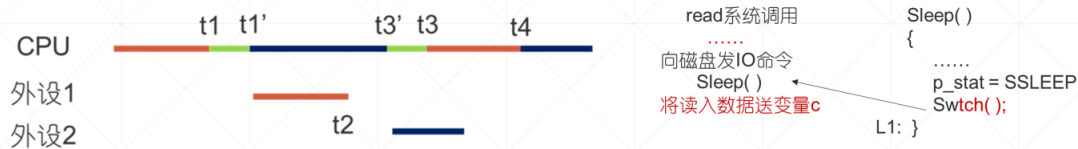
Part 4、Setpri() 计算进程执行应用程序时的优先数

```
void Process::SetPri()
{
    int priority;
    ProcessManager& procMgr = Kernel::Instance().GetProcessManager();

    priority = this->p_cpu / 16;
    priority += ProcessManager::PUSER + this->p_nice;

    if ( priority > 255 )
    {
        priority = 255;
    }
    if ( priority > procMgr.CurPri )
    {
        procMgr.RunRun++;
    }
    this->p_pri = priority;
}
```

t3' → t3 系统调用下半段



t3', 进程切换



恢复Swtch() 栈帧的ESP、EBP 函数返回, 建立地址映射关系

操作系统(拔尖班 / 国豪 2024)

t3, 模式切换



继续执行系统调用, 送磁盘文件数据至用户空间, 变量c

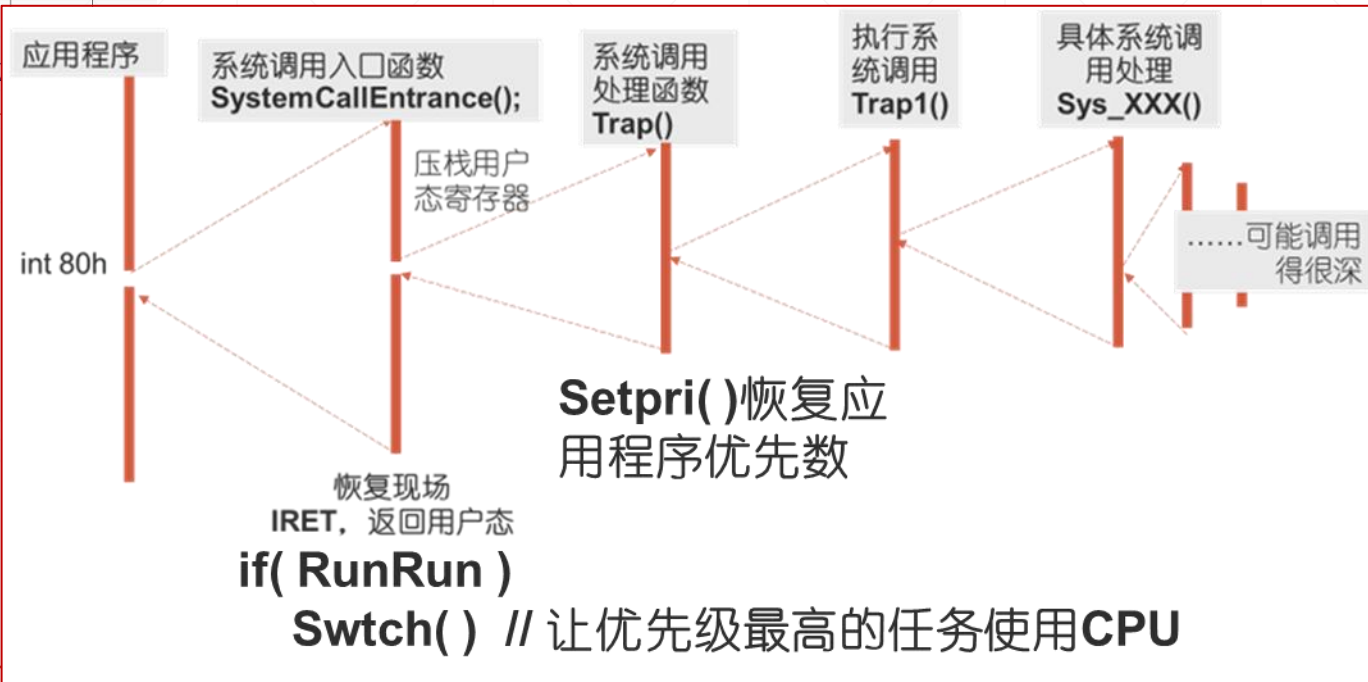
系统调用执行完毕 恢复用户态寄存器

继续执行应

同济大学计算机系 邓蓉

执行下半部时, 优先数是-50。所以, 返回用户态前要恢复应用程序优先数 (大于100的一个值)

系统调用返回前, 执行Setpri() 计算、重置优先数p_pri



Part 5、被信号打断的系统调用

Trap1()

{

1. u_error = User::NOERROR

2. u_intflg = 1

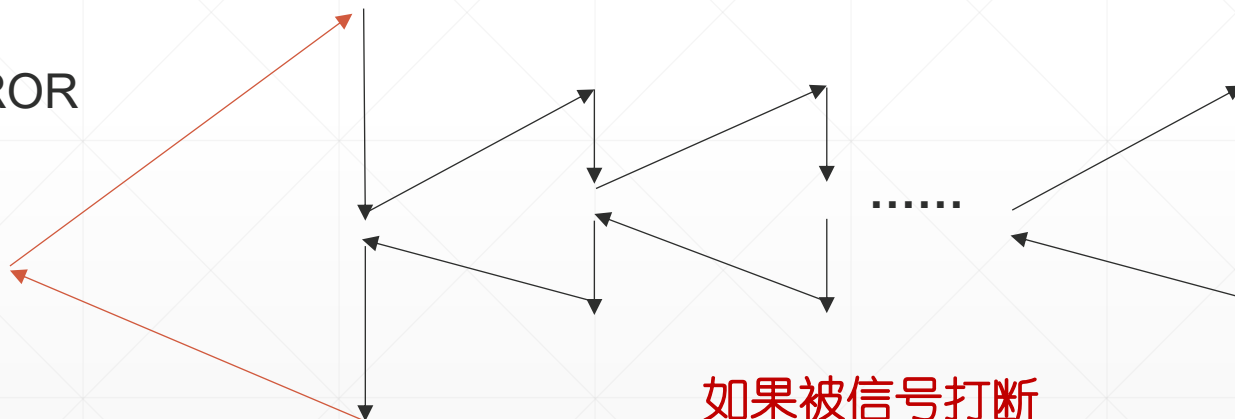
3. SaveU(u.u_qsav)

4. 调用系统调用入口函数

5. u_intflg = 0

}

Sys_***()



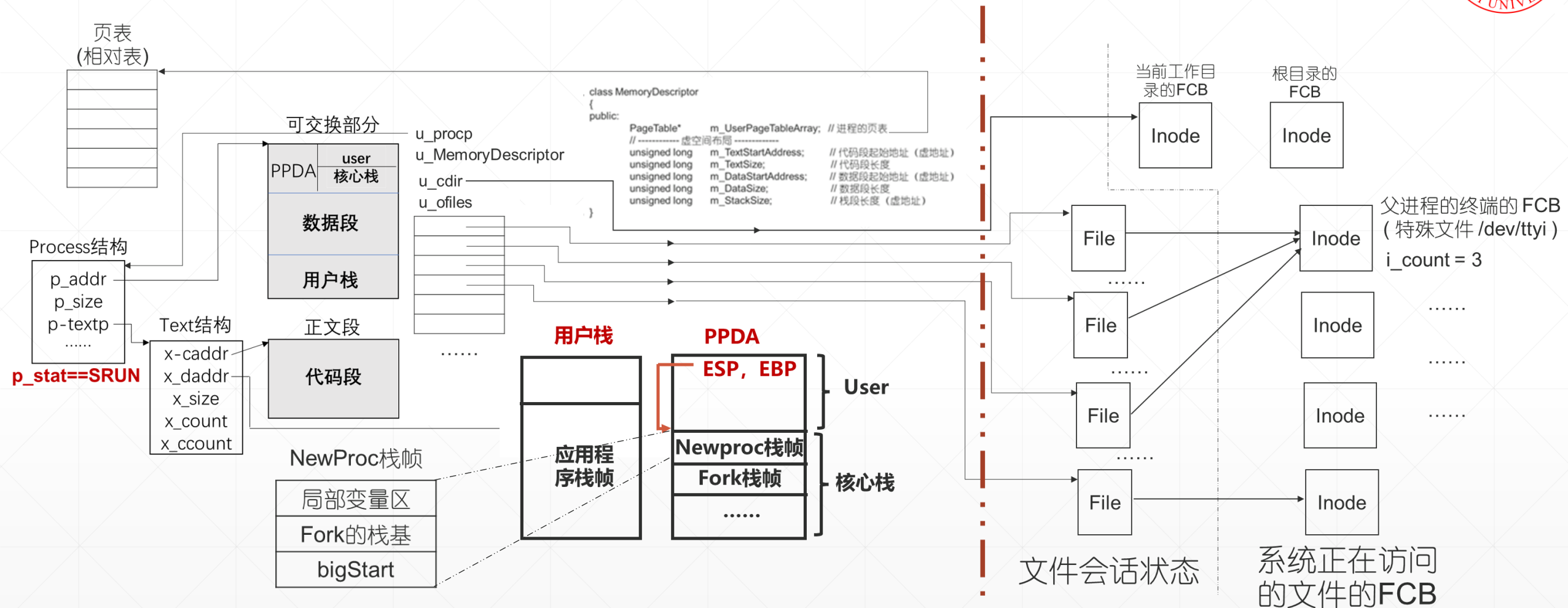
如果被信号打断
RetU(u.u_qsav)

Part 6、fork系统调用

- 创建子进程：分配内存，复刻父进程图像。具体而言：
 - 为其分配Process结构，PID，相对表 和 可交换部分
 - 复制父进程的 Process结构(p_uid, p_ttyp)，相对表 和 可交换部分
 - 共享父进程的Text结构 和 全部资源 (u_cdir, u_ofiles STDIN、STDOUT、STDERR)
- 子进程和父进程执行同一个应用程序。子进程创建成功后，
 - 父进程，fork系统调用返回，返回值是子进程的PID（一个正整数）；
 - 子进程，SRUN。被0#进程选中后，fork系统调用返回，返回值是0。

∴ 虽然父子进程执行的是同一个应用程序，程序员依然可以用fork返回值区分不同进程的执行分支。

父进程执行Newproc()函数，将自己的图像复制给子进程



子进程拥有可执行图像：SRUN，可被 Swtch() 调度



2#系统调用fork，源代码分析第2部分

int fork() // 1、用户空间钩子函数

```
{  
    int res;  
    __asm__ __volatile__ ( "int $0x80":"=a"(res):"a"(2));  
    if ( res >= 0 )  
        return res;  
    return -1;  
}
```

2、CPU硬件用中断号, 2, 查IDT & 跳转至系统调用总入口函数 SystemCallEntrance()

3、Trap() → Trap1() 驱动 fork系统调用的入口函数 void ProcessManager::Fork()



```
void ProcessManager::Fork()
```

```
{
```

```
    User& u = Kernel::Instance().GetUser();
```

```
    Process* child = NULL;;
```

```
    for ( int i = 0; i < ProcessManager::NPROC; i++ )
```

```
        if ( this->process[i].p_stat == Process::SNULL )
```

```
        {
```

```
            child = &this->process[i];
```

```
            break;
```

```
        } // 1、子进程有可用的Process结构 ?
```

```
    if ( child == NULL ) // 2、没有，不等
```

```
    {
```

```
        u.u_error = User::EAGAIN;
```

```
        return;
```

```
    }
```

```
        .....
```



```
.....
if ( this->NewProc() ) // 3、父进程创建子进程图像
{ // 新生的子进程，总有一天会被Swtch( )选中， Swtch( )返回 1。
    u.u_ar0[User::EAX] = 0;
    u.u_cstime = 0; //  $\Sigma$ 子进程的子进程的核心态运行时长
    u.u_stime = 0; // 子进程核心态运行时长
    u.u_cutime = 0; // 用户态运行时长
    u.u_utime = 0;
} ∴ 子进程 fork 系统调用的返回值是 0
else
{ // 子进程创建完毕后，父进程Newproc( )返回。返回值是0。
    u.u_ar0[User::EAX] = child->p_pid; ∴ 父进程 fork 系统调用的返回值是子进程的 PID
}
return;
}
```

int ProcessManager::NewProc()

```
{
```

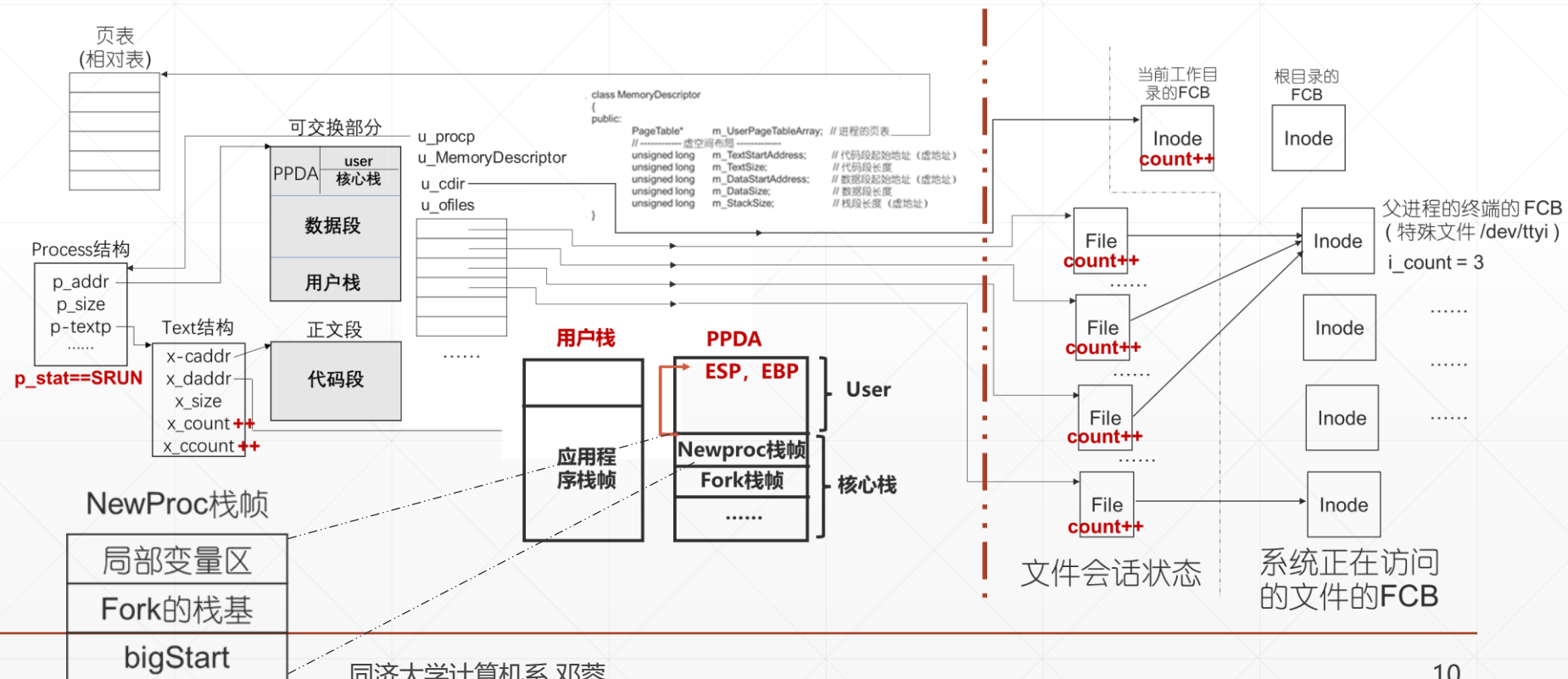
```
.....
current->Clone(*child); // 1、为子进程分配 Process 结构，复制父进程的
```

```
SaveU(u.u_rsav); // 2、将寄存器ESP、EBP的值存入父进程的User结构。这是制作可执行进程图像的重要步骤
```

```
3、无脑复制父进程相对表、可交换部分（相同的应用程序代码、数据和执行状态，相同的核心栈）
```

```
return 0
```

```
}
```



父进程创建子进程后，NewProc返回。EAX 是 0



```
.....
bigStart: if ( this->NewProc() )
{
L1:      u.u_ar0[User::EAX] = 0;
          u.u_cstime = 0;
          u.u_stime = 0;
          u.u_cutime = 0;
          u.u_utime = 0;
}
else
{
L2:      u.u_ar0[User::EAX] = child->p_pid;
}
return;
}
```

bigStart:

```
call NewProc
cmp EAX, 0
je L2
```

L1:

子进程执行的金色代码片段

L2:

父进程执行的紫色代码片段

父进程的核心栈

.....

if (this->NewProc())

{

L1:

u.u_ar0[User::EAX] = 0;

u.u_cstime = 0;

u.u_stime = 0;

u.u_cutime = 0;

u.u_ftime = 0;

}

else

{

L2:

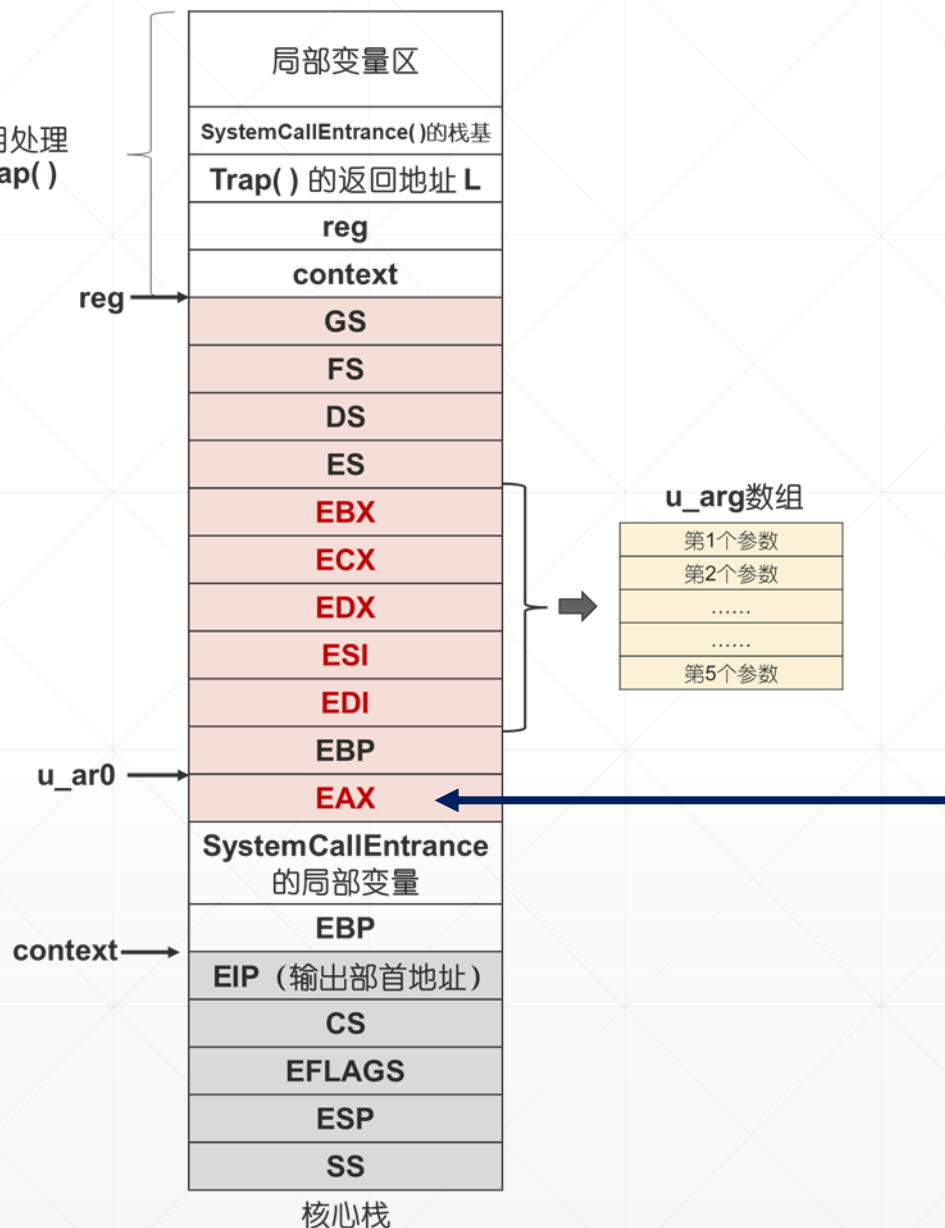
u.u_ar0[User::EAX] = child->p_pid;

}

return;

}

系统调用处理
程序 Trap()



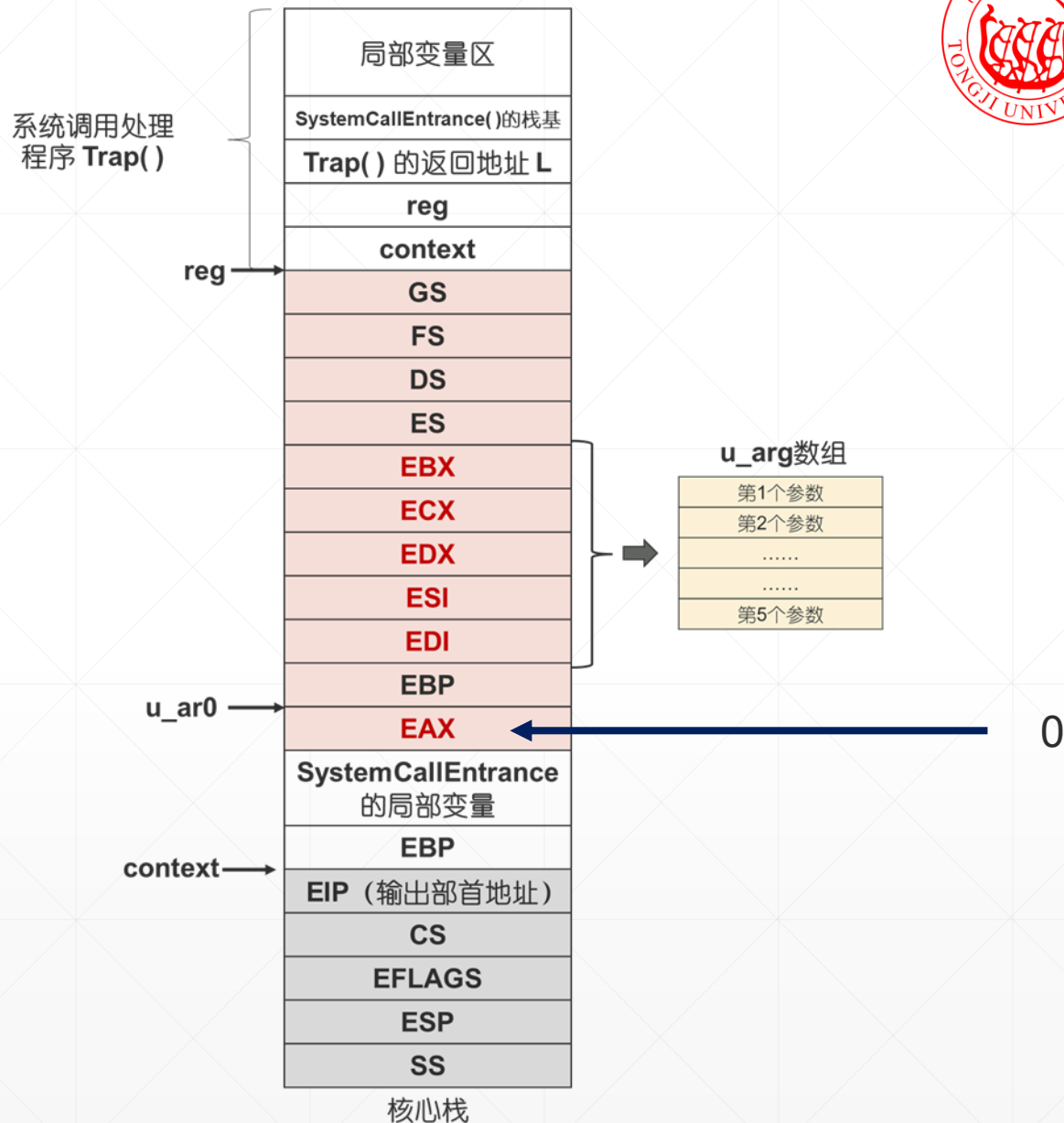
子进程PID



子进程的核心栈

```

.....
if ( this->NewProc() )
{
    L1:    u.u_ar0[User::EAX] = 0;
          u.u_cstime = 0;
          u.u_stime = 0;
          u.u_cutime = 0;
          u.u_utime = 0;
}
else
{
    L2:    u.u_ar0[User::EAX] = child->p_pid;
}
return;
}
    
```





Part 6、exec (11#系统调用) , 驱动应用程序

exec(应用程序的路径名, 为它提供的命令行参数);

- 进程执行exec系统调用, 加载目标应用程序。
 - Process结构和user结构不动。
 - 用新程序刷新用户空间
 - 代码段: 更新为新程序的代码段
 - 数据段: 更新为新程序的数据段
 - 用户栈: 底部是新程序的main栈帧, exec提供的命令行参数是main函数的入口参数
- 系统调用返回后, 进程从main()函数入口, 执行新程序。原先执行的程序, 从进程图像中消失, 不复存在。

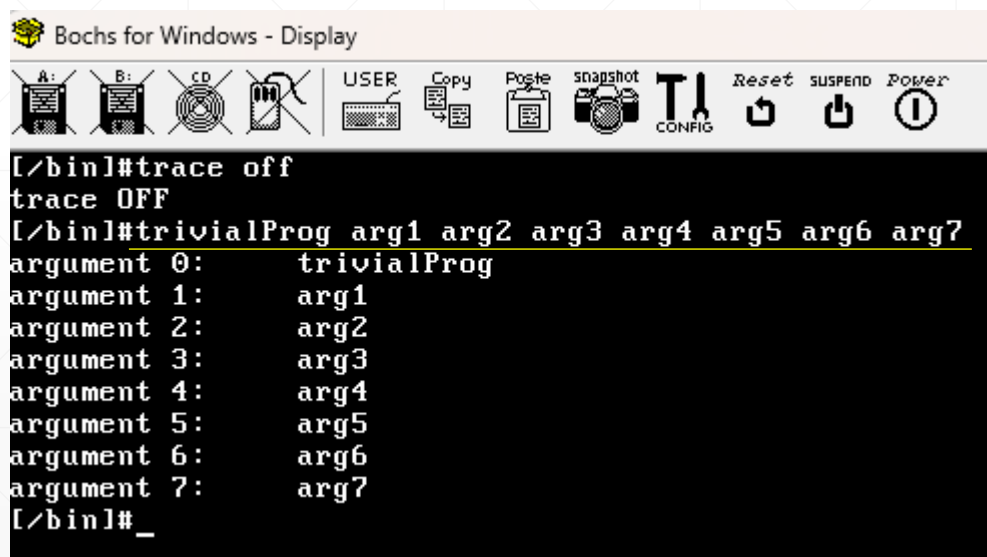
最简版 shell 程序

命令行中只有一个程序。
fork一个子进程，执行命令行中出现的程序。

```
While( )
{
    printf("$");           // 1、输出命令行提示符。
    scanf("%s", cmdLine);  // 2、读取用户输入的命令行
    parse(cmdLine, argv);  // 3、解析命令行，将命令行参数存入变长字符串数组 argv
    Swtch(argv[0]) {       // 4、执行内部命令
        case "cd":         shell进程改变当前工作目录
        case "logout":     shell进程终止
        ..... 其它内部命令 .....
    }
    // 5、执行外部命令。即，为命令行中出现的每个应用程序创建一个子进程，初始化进程图像
    i = fork( );           // shell进程创建1个子进程
    if(i)
        wait( );           // shell进程睡眠等待子进程终止（子进程独用tty与用户交互）
    else
        exec(argv[0], argv); // 子进程执行应用程序
}
```


一、exec系统调用的使用

回显命令行参数的trivialProg程序



```
Bochs for Windows - Display
A: B: CD USER Copy Paste snapshot T! CONFIG Reset SUSPEND Power
[/bin]#trace off
trace OFF
[/bin]#trivialProg arg1 arg2 arg3 arg4 arg5 arg6 arg7
argument 0: trivialProg
argument 1: arg1
argument 2: arg2
argument 3: arg3
argument 4: arg4
argument 5: arg5
argument 6: arg6
argument 7: arg7
[/bin]#_
```

```
#include <stdio.h>
#include <stdlib.h>
```

```
int main1(int argc, char *argv[])
{
    int i;
    for(i = 0; i < argc; i++)
        printf("argument %d:\t%s\n", i, argv[i]);
    exit(0);
}
```



tryExec, 启动trivialProg, 为其提供4个命令行参数

```
#include <stdio.h>
#include <stdlib.h>
#include <sys.h>
```

```
char *argv[]={“trivialProg”,“arg1”,“arg2”,“arg3”};    // 命令行参数
```

```
int main1( )
```

```
{
```

```
    int pid;
    int exitCode;
```

```
    pid = fork();    // 1、创建子进程
```

```
    if( pid == 0)
```

```
        execv(argv[0],argv);    // 2、子进程执行应用程序trivialProg
```

```
    else
```

```
        wait(&exitCode);    // 3、父进程入睡，等待子进程终止
```

```
    printf(“The end of tryExec.\n”);
```

```
}
```

A screenshot of a Bochs for Windows - Display window. The window title is "Bochs for Windows - Display". The display area shows a command prompt with the following text: [/]#cd bin, [/bin]#tryExec, argument 0: trivialProg, argument 1: arg1, argument 2: arg2, argument 3: arg3, The end of tryExec., [/bin]#_. The window has a toolbar with icons for A:, B:, CD, USER, Copy, and Paste.

作业：修改tryExec.c，写一个简版shell程序myShell.c



最简版 shell 程序

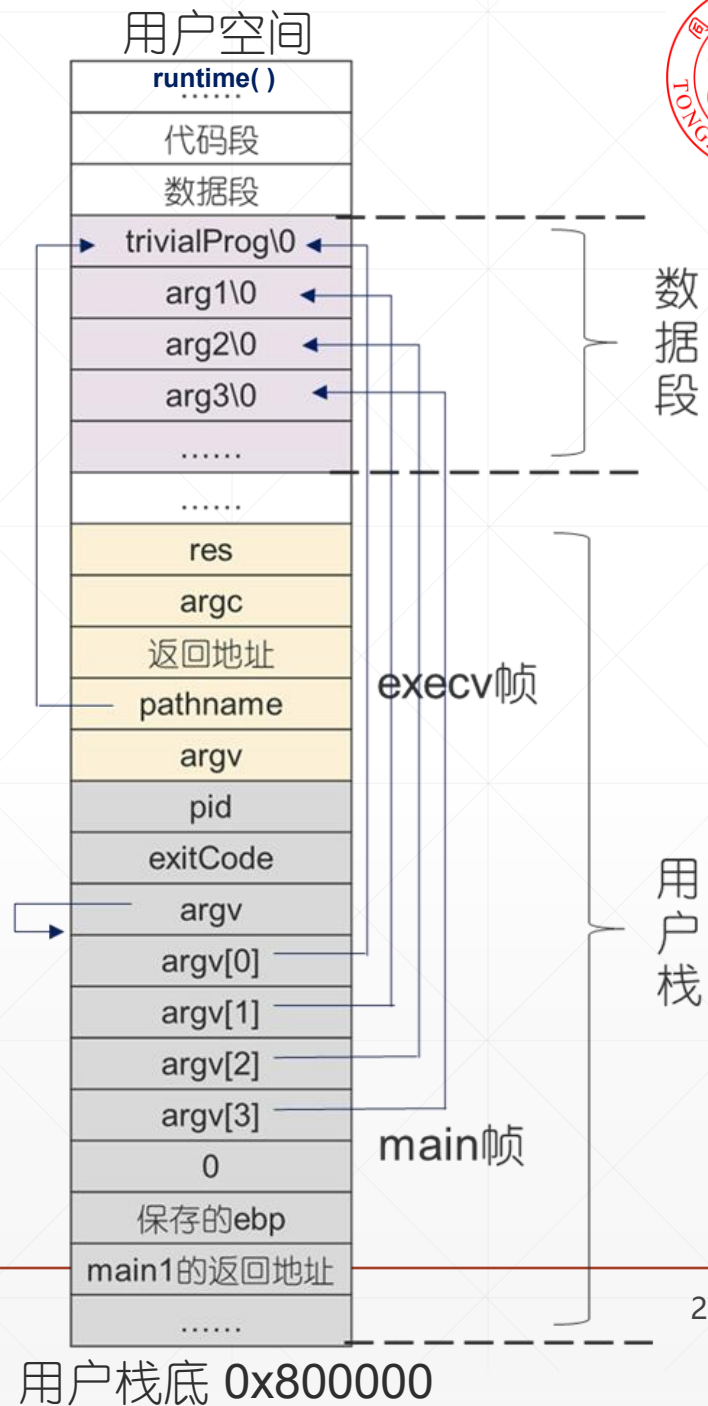
命令行中只有一个程序。
fork一个子进程，执行命令行中出现的程序。

```
While( )
{
    printf("$");           // 1、输出命令行提示符。
    scanf("%s", cmdLine);  // 2、读取用户输入的命令行
    parse(cmdLine, argv);  // 3、解析命令行，将命令行参数存入变长字符串数组 argv
    Switch(argv[0]) {      // 4、执行内部命令
        case "cd":         shell进程改变当前工作目录
        case "logout":     shell进程终止
        ..... 其它内部命令 .....
    }
    // 5、执行外部命令。即，为命令行中出现的每个应用程序创建一个子进程，初始化进程图像
    i = fork( );           // shell进程创建1个子进程
    if(i)
        wait( );           // shell进程睡眠等待子进程终止（子进程独用tty与用户交互）
    else
        exec(argv[0], argv); // 子进程执行应用程序
}
```



二、库函数 `execv`

```
int execv(char *pathname, char *argv[])
{
    int res;
    int argc = 0;
    while(argv[argc] != 0) // 1、清点命令行参数的数量
        argc++;
    __asm__ volatile ( "int $0x80": "=a"(res):
        "a"(11), "b"(pathname), "c"(argc), "d"(argv)); // 2、执行系统调用
    if ( res >= 0 )
        return res;
    return -1;
}
```



三、exec主要执行步骤

- 目录搜索、找到目标应用程序，确保进程有执行权限
- 收集命令行参数
- 清除正文段、数据段（可交换部分留PPDA）
- 读程序头，获得所有逻辑段长度。确保8M字节用户空间能够容纳用户空间所有逻辑段
- 重置内存描述符和相对表
- 构造正文段
- 构造可交换部分，用户栈底建立main栈帧
- 更新核心栈底 第1、第2现场保护区，设置执行应用程序所需的用户态寄存器初值
- IRET返回用户态后。执行虚地址0处的函数runtime()，后者驱动应用程序的主函数main()

四、可执行文件格式 (Unix V6++ , PE格式)

UNIX V6++操作系统的应用程序是PE (Portable Executable) 格式的。一个PE可执行文件由4部分组成:

- (1) 程序头
 - DOS头, 数据结构 ImageDosHeader
 - NT头, 数据结构 ImageNTHeader。这是重定位表
- (2) 段表, ImageSectionHeader结构数组
- (3) 逻辑段的初值
 - 代码段 (应用程序的函数)
 - 数据段 (全局变量的初值)
 - 只读数据段 (常量的值)
- (4) 符号表和其它与调试相关的信息

可执行文件

程序头
段表
代码段
数据段
只读数据段
符号表.....



struct ImageDosHeader // DOS头

```
{.....
    unsigned short  e_lfarlc;    // NT头的起始偏移量
.....};
```

```
struct ImageNTHHeader // NT头
```

```
{
    unsigned long Signature; // magic number: 0x00004550, 字符串“PE\0\0”的ASCII码
    ImageFileHeader FileHeader;
    ImageOptionalHeader32 OptionalHeader;
};
```

```
        unsigned long    ImageBase;
        unsigned long    SectionAlignment;
```

```
struct ImageFileHeader  
{  
    unsigned short Machine;  
    unsigned short NumberOfSections;  
    unsigned long TimeDateStamp;  
    unsigned long PointerToSymbolTable;  
    unsigned long NumberOfSymbols;  
    unsigned short SizeOfOptionalHeader;  
    unsigned short Characteristics;  
};
```

```
struct ImageOptionalHeader32  
{  
    //  
    // Standard fields  
  
    unsigned short Magic;  
    unsigned char MajorVersion;  
    unsigned char MinorVersion;  
    unsigned long Signature;  
    unsigned long ImageBase;  
    unsigned long SectionPointer;  
    unsigned long FilePointer;  
    unsigned long BaseOfData;  
    unsigned long BaseOfCode;
```

```

unsigned long    ImageBase;
unsigned long    SectionAlignment;
unsigned long    FileAlignment;
unsigned short   MajorOperatingSystemVersion;
unsigned short   MinorOperatingSystemVersion;
unsigned short   MajorImageVersion;

```

ImageBase: 映像文件装入进程图像时, 起始地址

AddressOfEntryPoint: 程序入口地址

BaseOfCode: 代码段起始地址 = BaseOfCode + ImageBase

SizeOfCode: 代码段尺寸

BaseOfData: 数据段起始地址 = BaseOfData + ImageBase

SizeOfInitializedData: 数据段尺寸

SizeOfUninitializedData: BSS段尺寸

```

unsigned long    SizeOfHeapCommit;
unsigned long    LoaderFlags;
unsigned long    NumberOfRvaAndSizes;
ImageDataDirectory DataDirectory[IMAGE_NUMBEROF_DIRECTORY_ENTRIES];

```




```
struct ImageSectionHeader           // 段表中的元素。每个逻辑段一个段头，登记逻辑段的属性
{
    char    Name[8];
    union {
        unsigned long    PhysicalAddress;
        unsigned long    VirtualSize; // 虚空间中，本段长度
    } Misc;
    unsigned long    VirtualAddress; // 虚空间中，本段起始地址 = ImageBase+ VirtualAddress
    unsigned long    SizeOfRawData; // 可执行文件中，本段的长度
    unsigned long    PointerToRawData; // 可执行文件中，本段的起始偏移量
    unsigned long    PointerToRelocations;
    unsigned long    PointerToLinenumbers;
    unsigned short    NumberOfRelocations;
    unsigned short    NumberOfLinenumbers;
    unsigned long    Characteristics; // 段属性，代码段0x00000002h，带初值的数据段0x00000004h，
                                     BSS段0x00000008h
};
```

```
116 class PEParse
117 {
118 public:
119     static const unsigned int TEXT_SECTION_IDX = 0;
120     static const unsigned int DATA_SECTION_IDX = 1;
121     static const unsigned int RDATA_SECTION_IDX = 2;
122     static const unsigned int BSS_SECTION_IDX = 3;
123     static const unsigned int IDATA_SECTION_IDX = 4;
124 }
```

应用程序的逻辑段：

- 0#，代码段。程序运行需要调用的所有子程序的机器码。
- 1#，数据段。可执行文件中存有全局变量初值。
- 2#，只读数据段。可执行文件中存有常量的值。
- 3#，bss段。bss变量初值为0，可执行文件中没必要存初值。
- 4#，输入数据段。被引用的其它模块定义的变量。（不完全确定）

五、exec 加载应用程序的细节

- 读出目标可执行程序program header 和 section headers。
- 写内存描述符 **MemoryDescriptor**
- EstablishUserPageTable() 写相对表
- 为代码段分配内存
 - 可与其它进程共享正文段，跳过
 - 不能。AllocMemory()内存分配
- Expand() [1] AllocMemory()为可交换部分分配内存
[2] 复制PPDA区至新空间顶部
[3] MapToPageTable()刷新系统页表，映射为进程分配的新的内存块。

```
class MemoryDescriptor
{
    PageTable*      m_UserPageTableArray; /* 相对表 */
    unsigned long   m_TextStartAddress;    /* 代码段起始地址 */
    unsigned long   m_TextSize;            /* 代码段长度 */
    unsigned long   m_DataStartAddress;    /* 数据段起始地址 */
    unsigned long   m_DataSize;            /* 数据段长度 */
    unsigned long   m_StackSize;           /* 栈段长度 */
}
```

- 初始化进程的所有逻辑段

代码段：清0，从磁盘文件读入
数据段：清0，从磁盘文件读入全局变量的初值
只读数据段：清0，从磁盘文件读入常量的值
bss段：清0
堆栈段：什么也不做（局部变量初始值是乱码）

六、源码分析

预备

```
//      系统调用处理函数
void SystemCall::Trap(struct pt_regs* regs, struct pt_context* context)
{
    .....
    u.u_dirp = (char *)u.u_arg[0]; // 目标程序的文件名
    u.u_arg[4] = (int)context;      // 第一现场保护区的首地址
    .....
    间接函数调用 Sys_Exec()
    .....
}

//      exec系统调用的入口函数
int SystemCall::Sys_Exec()
{
    ProcessManager& procMgr = Kernel::Instance().GetProcessManager();
    procMgr.Exec();
    return 0; /* GCC likes it ! */
}
```



```
void ProcessManager::Exec()
{
    Inode* pInode; // 目标应用程序的文件控制块
    Text* pText;    // 管理目标应用程序的Text结构
    User& u = Kernel::Instance().GetUser();    // 现运行进程的用户
    FileManager& fileMgr = Kernel::Instance().GetFileManager(); // open, read, write...系统调用
    UserPageManager& userPgMgr = Kernel::Instance().GetUserPageManager(); // 内存分配
    KernelPageManager& kernelPgMgr = Kernel::Instance().GetKernelPageManager();
        // 内核使用的动态内存
    BufferManager& bufMgr = Kernel::Instance().GetBufferManager(); // 管理磁盘缓存块，缓存块
                                                                    中是系统最近使用过的磁盘扇区副本
}
```

Exec系统调用开销巨大。

涉及：文件管理，内存分配、释放，会用磁盘高速缓存。需要借用页表区临时存放应用程序的命令行参数。

所以，同时执行的**Exec**系统调用，数量不能超过3个。



1、目录搜索目标程序，确认现运行进程的执行权限

`plnode = fileMgr.Namel(FileManager::NextChar, FileManager::OPEN);` // 1、目录搜索，找目标程序，把
..... 文件系统里没有目标程序，失败返回..... 它的文件控制块 (**Inode**) 复制到内存

```
while( this->ExeCnt >= NEXEC )           // 2、已经有3个Exec在跑了，等等
{
    u.u_procp->Sleep((unsigned long)&ExeCnt, ProcessManager::EXPRI);
}
this->ExeCnt++;
```

```
if ( fileMgr.Access(plnode, Inode::IEXEC) || (plnode->i_mode & Inode::IFMT) != 0 )
{
    // 3、确认进程有执行权限 (文件控制块中的RWXRWXRWX, p_uid对应的 Xbit 为1)
    fileMgr.m_InodeTable->IPut(plnode);
    if ( this->ExeCnt >= NEXEC )
    {
        WakeUpAll((unsigned long)&ExeCnt);
    }
    this->ExeCnt--;
    return;
}
```

} 出错处理分支

2、PE对象读程序头

```
PEParser parser;
```

```
if ( parser.HeaderLoad(pInode)==false )
{
    fileMgr.m_InodeTable->IPut (pInode);
    return;
}
```



Signature // 可执行文件的签名
 NumberOfSections // 段的数量
 ImageBase // 虚地址 = 地址 + ImageBase
 AddressOfEntryPoint // 入口地址

```
bool PEParser::HeaderLoad(Inode* p_inode)
{
```

```
    ImageDosHeader dos_header;
    User& u = Kernel::Instance().GetUser();
    KernelPageManager& kpm = Kernel::Instance().GetKernelPageManager();
```

```
    /*读取dos header*/
```

```
    u.u_IOParam.m_Base = (unsigned char*)&dos_header;
    u.u_IOParam.m_Offset = 0;
    u.u_IOParam.m_Count = 0x40;
    p_inode->ReadI();
```

读可执行文件，将偏移量为
 [0,0x40)的 DOS头 存进数
 据结构 dos_header

```
    /*读取nt_Header*/
```

```
    u.u_IOParam.m_Base = (unsigned char*)(&this->ntHeader);
    u.u_IOParam.m_Offset = dos_header.e_lfanew;
    u.u_IOParam.m_Count = ntHeader_size;
    p_inode->ReadI();
```

读 NT头，存进 ntHeader

```
    if ( ntHeader.Signature!=0x00004550 )
```

确认这是PE格式的可执行文件

```
    {
        return false;
    }
```

```
    sectionHeaders = (ImageSectionHeader*)(kpm.AllocMemory(PageManager::PAGE_SIZE * 2) + 0xC0000000);
    u.u_IOParam.m_Base = (unsigned char*)sectionHeaders;
    u.u_IOParam.m_Offset = dos_header.e_lfanew + ntHeader_size;
    u.u_IOParam.m_Count = section_size * ntHeader.FileHeader.NumberOfSections;
    p_inode->ReadI();
```

AllocMemory页表区，为段表sectionHeaders

分配8K动态内存，读段表



PE对象，计算进程，正文段、数据段、用户栈的首地址和长度，计算entry point的地址

```
this->TextAddress =  
    ntHeader.OptionalHeader.BaseOfCode + ntHeader.OptionalHeader.ImageBase;  
this->TextSize =  
    ntHeader.OptionalHeader.BaseOfData - ntHeader.OptionalHeader.BaseOfCode;  
  
this->DataAddress =  
    ntHeader.OptionalHeader.BaseOfData + ntHeader.OptionalHeader.ImageBase;  
this->DataSize = this->sectionHeaders[this->IDATA_SECTION_IDX].VirtualAddress - ntHeader.OptionalHeader.BaseOfData;  
  
StackSize = ntHeader.OptionalHeader.SizeOfStackCommit;  
HeapSize = ntHeader.OptionalHeader.SizeOfHeapCommit;  
  
EntryPointAddress = ntHeader.OptionalHeader.AddressOfEntryPoint +  
    ntHeader.OptionalHeader.ImageBase;  
  
return true;  
}
```

section 顺序为 .text->.data->.rdata->.bss->.idata

段表，没有得到很好的利用

段表**SectionHeaders**，管理应用程序的段，有多少个段就有多少个元素。
每个元素是一个**Section Header** (**ImageSectionHeader**)，登记逻辑段属性。

```
struct ImageSectionHeader
{
    char    Name[8];
    union {
        unsigned long    PhysicalAddress;
        unsigned long    VirtualSize; 虚空间中，段长
    } Misc;
    unsigned long    VirtualAddress; 虚空间首地址（要加 ImageBase）
    unsigned long    SizeOfRawData; 文件中的起始偏移量和长度
    unsigned long    PointerToRawData;
    unsigned long    PointerToRelocations;
    unsigned long    PointerToLinenumbers;
    unsigned short    NumberOfRelocations;
    unsigned short    NumberOfLinenumbers;
    unsigned long    Characteristics;
};
```


3、写内存描述符

```
u.u_MemoryDescriptor.m_TextStartAddress = parser.TextAddress;
u.u_MemoryDescriptor.m_TextSize = parser.TextSize;

u.u_MemoryDescriptor.m_DataStartAddress = parser.DataAddress;
u.u_MemoryDescriptor.m_DataSize = parser.DataSize;

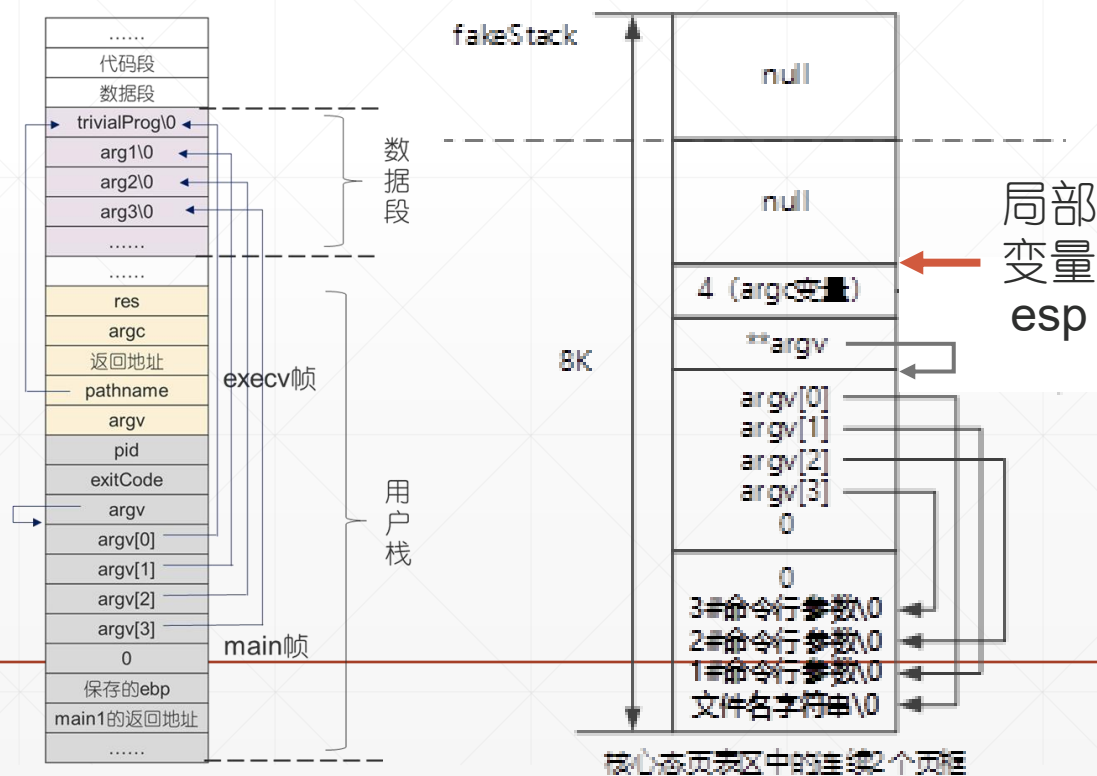
u.u_MemoryDescriptor.m_StackSize = parser.StackSize;

if ( parser.TextSize + parser.DataSize + parser.StackSize + PageManager::PAGE_SIZE >
    MemoryDescriptor::USER_SPACE_SIZE - parser.TextAddress)
{
    fileMgr.m_InodeTable->IPut(plnode);
    u.u_error = User::ENOMEM;
    return;
} // 程序太大，Unix V6++进程虚空间装不下。Exec失败返回。
```

4、构造初始用户栈(压栈保存新程序的命令行参数，形成main栈帧)

int allocLength = (parser.StackSize + PageManager::PAGE_SIZE * 2 - 1) >> 13 << 13; // 1、计算初始用户栈大小
 unsigned long fakeStack = kernelPgMgr.AllocMemory(allocLength); // 2、页表区分配连续2个页框

~ Line 576 // 3、从用户空间复制命令行参数，构造用户栈底main栈帧（实参区）。使用关键局部变量esp，软件模拟push指令执行的硬件压栈操作。图中红色箭头是操作完成后，指针esp的位置。



5、释放为用户空间分配的物理内存

```
/* 释放原进程图像的共享正文段，数据段，堆栈段 */
```

```
if ( u.u_procp->p_textp != NULL )
```

```
{
```

```
    u.u_procp->p_textp->XFree();
```

```
    u.u_procp->p_textp = NULL;
```

```
}
```

```
u.u_procp->Expand(ProcessManager::USIZE);
```

// 正文段引用计数减1。减至0，释放内存。

// 内存中，缩小进程的可交换部分，只留PPDA区。



6、构造新的Text结构

```
pText = NULL;

for ( int i = 0; i < ProcessManager::NTEXT; i++ )
{
    if ( NULL == this->text[i].x_iptr )
    {
        if ( NULL == pText )
        {
            pText = &(this->text[i]); // 记住第1个未被占用的位置
        }
    }
    else if ( plnode == this->text[i].x_iptr ) // 目标程序正文段的text已存在
    {
        this->text[i].x_count++; // 共享正文段
        this->text[i].x_ccount++;
        u.u_procp->p_textp = &(this->text[i]);
        pText = NULL; // 打一个标识
        break;
    }
}
```



7、为正文段和可交换部分分配内存

```
int sharedText = 0;

if ( NULL != pText ) // 为代码段分配物理内存 & 初始化Text结构
{
    plnode->i_count++;
    pText->x_ccount = 1;
    pText->x_count = 1;
    pText->x_iptr = plnode;
    pText->x_size = u.u_MemoryDescriptor.m_TextSize;
    pText->x_caddr = userPgMgr.AllocMemory(pText->x_size);
    pText->x_daddr = Kernel::Instance().GetSwapperManager().AllocSwap(pText->x_size);
    u.u_procp->p_textp = pText;
}
else
{
    pText = u.u_procp->p_textp;
    sharedText = 1;
}

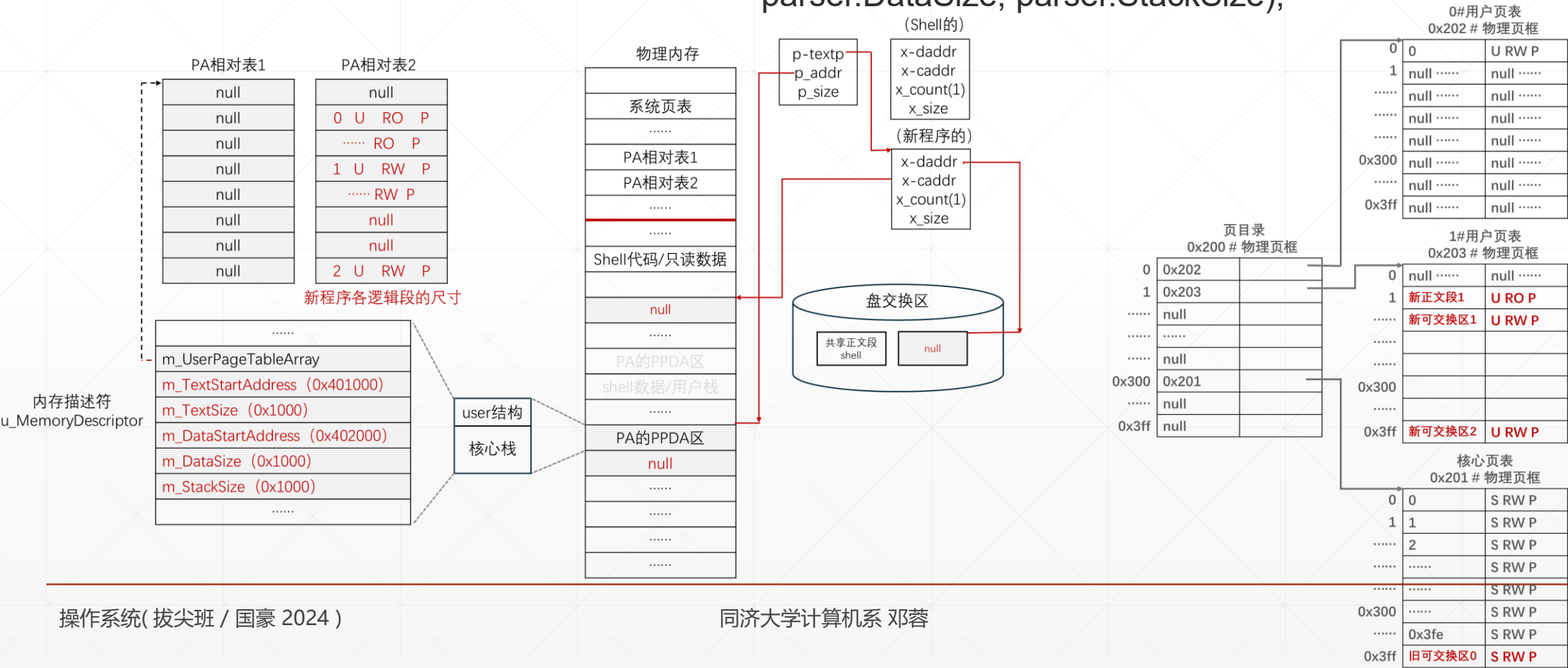
unsigned int newSize = ProcessManager::USIZE + u.u_MemoryDescriptor.m_DataSize + .....m_StackSize;
u.u_procp->Expand(newSize); // 为可交换部分分配物理内存（PPDA+数据+堆栈），复制原PPDA区
```



8、重建相对表、内存描述符 和 用户页表

按新的进程尺寸刷新虚空间 & 建立新的虚实地址映射关系，映射为用户空间分配的新的物理内存

```
u.u_MemoryDescriptor.EstablishUserPageTable(parser.TextAddress, parser.TextSize, parser.DataAddress, parser.DataSize, parser.StackSize);
```



9、初始化正文段 和 可交换部分 1 对所有逻辑段清0

```
parser.Relocate(pInode, sharedText);
```

```
for (; i <= this->BSS_SECTION_IDX; i++) // 逐段清0
```

```
{
```

```
ImageSectionHeader* sectionHeader = &(this->sectionHeaders[i]);
int beginVM = sectionHeader->VirtualAddress + ntHeader.OptionalHeader.ImageBase;
int size = ((sectionHeader->Misc.VirtualSize + PageManager::PAGE_SIZE - 1)>>12)<<12;
```

```
int j;
```

```
for (j=0; j<size; j++)
```

```
{
```

```
    unsigned char* b =(unsigned char*)(j + beginVM);
    *b = 0;
```

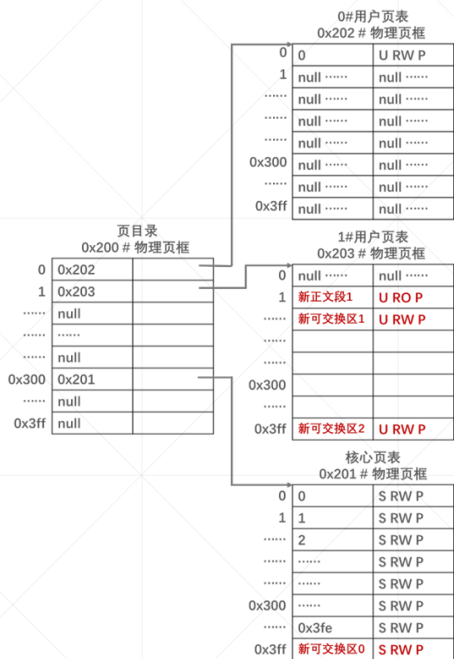
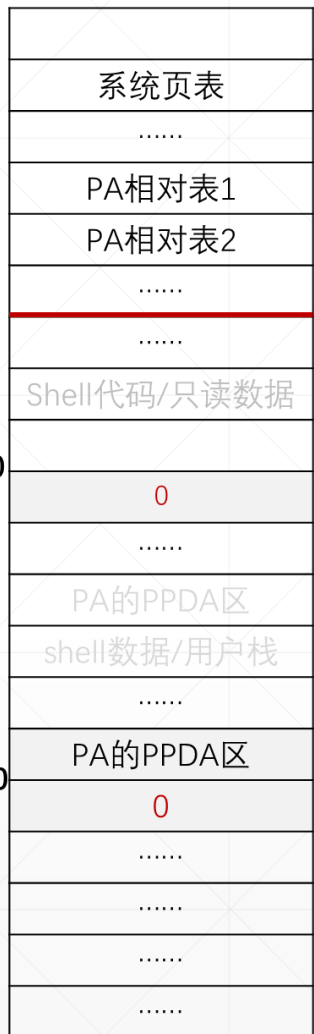
```
}
```

```
}
```

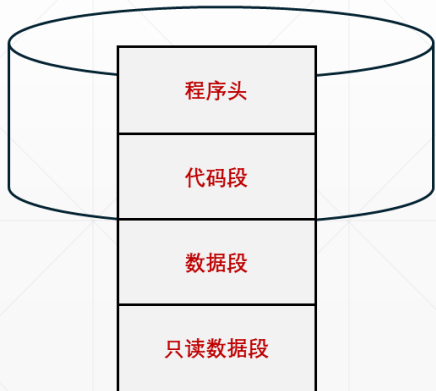
细节：跳过共享正文段

写共享正文段期间，PTE设可写标识RW。完成以后，改回RO

物理内存



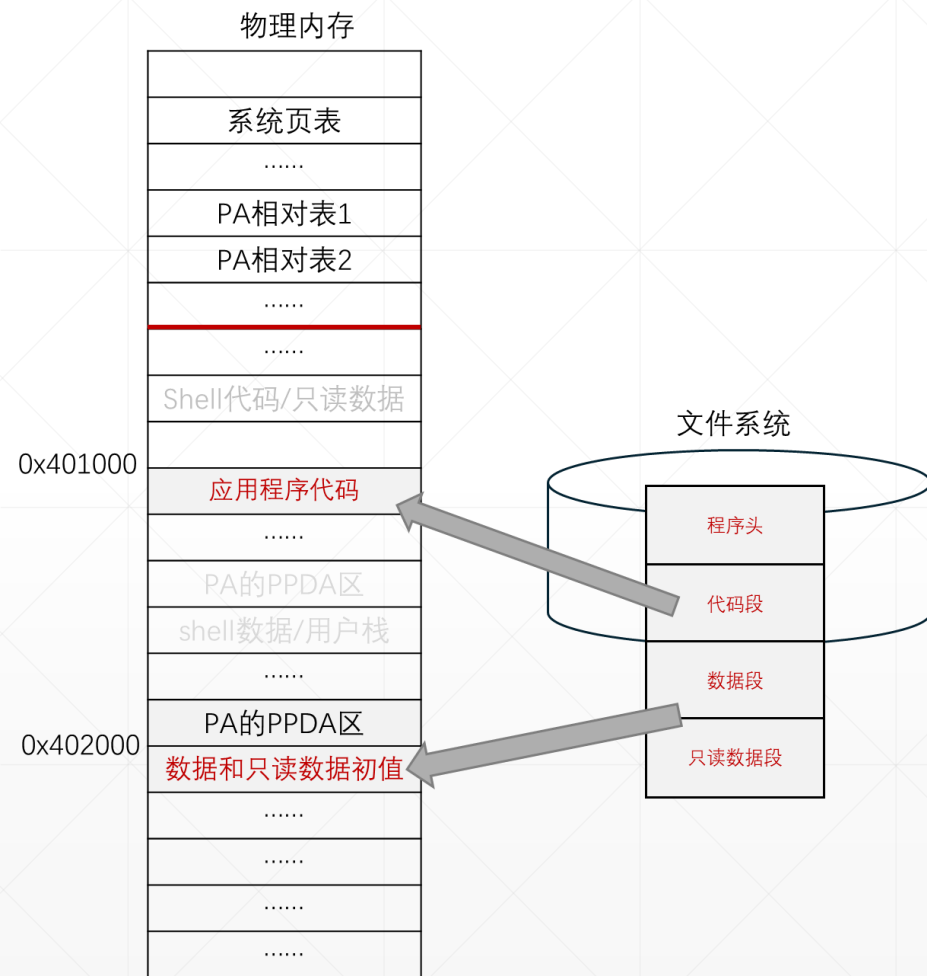
文件系统



9、初始化正文段 和 可交换部分

2 读入正文段、数据段和只读数据段

```
parser.Relocate(pInode, sharedText);
```



```
for ( ; i < this->BSS_SECTION_IDX; i++ ) // 从可执行文件读逻辑段初值
{
```

```
    ImageSectionHeader* sectionHeader = &(this->sectionHeaders[i]);
    srcAddress = sectionHeader->PointerToRawData; // 逻辑段在可执行文件中的偏移量
    desAddress = this->ntHeader.OptionalHeader.ImageBase + sectionHeader->VirtualAddress; // 虚空间的首地址
```

```
    u.u_IOParm.m_Base = (unsigned char*)desAddress;
    u.u_IOParm.m_Offset = srcAddress;
    u.u_IOParm.m_Count = sectionHeader->Misc.VirtualSize; // 段长度
    p_inode->ReadI(); // 读文件
```

```
}
```



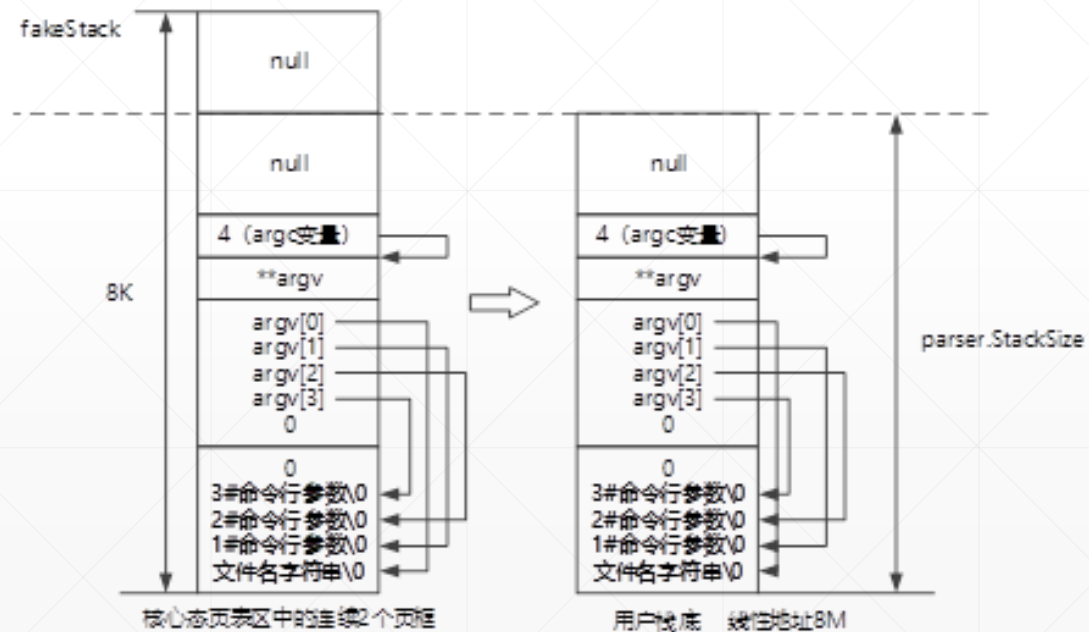
10、共享正文段在盘交换区上留复本

```
/* .text段在swap分区上留复本 */
if(sharedText == 0)
{
    u.u_procp->p_flag |= Process::SLOCK;
    // 启动磁盘IO，将物理内存中首地址为x_caddr，长度为x_size的数据块写进磁盘，起始扇区号是x_daddr
    bufMgr.Swap(pText->x_daddr, pText->x_caddr, pText->x_size, Buf::B_WRITE);
    // IO完成后，Swap返回
    u.u_procp->p_flag &= ~Process::SLOCK;
}
```

11、将fakeStack中暂存的main栈帧，复制到用户栈底

```
Utility::MemCopy(fakeStack + allocLength - parser.StackSize | 0xC0000000,
                 MemoryDescriptor::USER_SPACE_SIZE - parser.StackSize, parser.StackSize);
kernelPgMgr.FreeMemory(allocLength, fakeStack); // 释放fakeStack占用的页框
```

```
fileMgr.m_InodeTable->IPut(pInode); // 释放内存Inode
```



Done! 进程图像, 刷新完毕

12、创建应用程序执行的初始环境

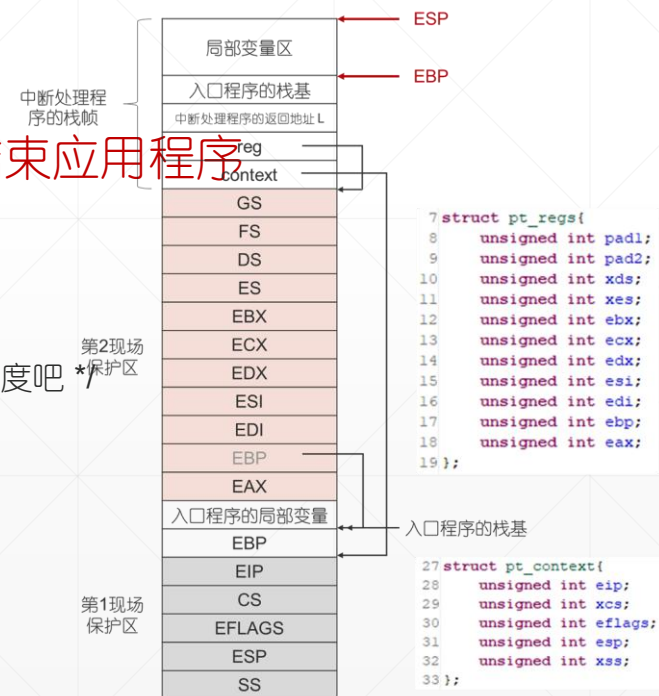
```
if ( this->ExeCnt >= NEXEC ) // 释放exec执行权限的配额
{
    WakeUpAll((unsigned long)&ExeCnt); // 唤醒等待执行exec系统调用的进程
}
this->ExeCnt--;
```

```
for (int i = 0; i < u.NSIG ; i++)
{
    u.u_signal[i] = 0; // 设置信号的默认处理方式：杀死进程，提前结束应用程序
}
```

```
// /* 清0所有通用寄存器 */
// for (int i = User::EAX - 4; i < User::EAX - 4*7 ; i = i - 4)
// {
```

~~u.u_ar0[i] = 0; /* 下标写成 User::EAX + i 可读性要强一些，但是运算速度慢了。就小抠，追求速度吧 */~~

```
// }
// for (int i = User::EAX - 2; i > User::EAX - 7 ; i--)
// {
//     u.u_ar0[i] = 0;
// }
```

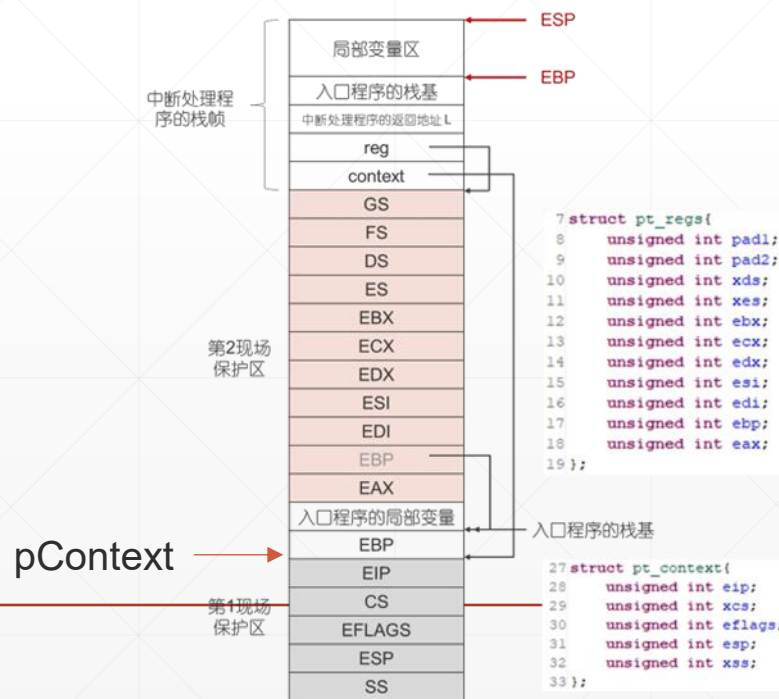




```
// 将main函数的入口地址写入核心栈现场保护区中的用户态寄存器EAX
u.u_ar0[User::EAX] = parser.EntryPointAddress;
```

```
struct pt_context* pContext = (struct pt_context *)u.u_arg[4];
pContext->eip = 0x00000000; // exec系统调用返回用户态后, 执行 runtime() 函数驱动应用程序
pContext->xcs = Machine::USER_CODE_SEGMENT_SELECTOR; // 用户代码段的段选择子
pContext->eflags = 0x200; // 应用程序应该运行在开中断状态下
pContext->esp = esp; // 用户栈顶
pContext->xss = Machine::USER_DATA_SEGMENT_SELECTOR; // 用户数据段的段选择子
```

```
}
```



七、返回用户态，执行 runtime() 函数

```
extern "C" void runtime()
```

```
{
```

```
    __asm__ __volatile__("
```

```
        leave; \
```

```
        movl %%esp, %%ebp; \
```

```
        call *%%eax; \
```

```
        movl $1, %%eax; \
```

```
        movl $0, %%ebx; \
```

```
        int $0x80::);
```

```
}
```

\ 调用应用程序的main()函数。

} main()函数返回后，执行系统调用 exit(0)，进程终止。